COLEÇÃO BANCADA · VOLUME 3

```
>>> import pathlib, csv
>>> automatize(o_que_se_repete)
```



Guia prático de Python

Do primeiro script à automação das tarefas do dia a dia

MARINA TOLEDO

2ª edição · 2026

Apresentação

Este guia ensina Python de forma prática, do primeiro script à automação de tarefas do dia a dia, com exemplos curtos que você pode copiar, rodar e adaptar ao seu trabalho. A ideia não é cobrir a linguagem inteira, e sim o pedaço dela que resolve oitenta por cento dos problemas reais: ler e escrever arquivos, organizar dados, repetir tarefas chatas sem errar e escrever código que outra pessoa consiga manter.

Cada capítulo termina com um exercício. Faça todos: programação se aprende digitando, errando e lendo a mensagem de erro com calma. Ao final, você terá montado um pequeno conjunto de ferramentas pessoais que economizam algumas horas por semana.

Para quem é este guia

Para quem já usa planilhas com desenvoltura e sente que poderia automatizar parte do trabalho, para estudantes que querem uma primeira linguagem e para desenvolvedores de outras linguagens que precisam ler e escrever Python com segurança em poucos dias.

1. Preparando o ambiente

Instale a versão mais recente do Python 3 pelo site oficial ou pelo gerenciador de pacotes do seu sistema. No Windows, marque a opção que adiciona o Python ao PATH. Depois, confira no terminal:

```
python --version
python -m pip --version
```

Crie sempre um **ambiente virtual** por projeto. Ele isola as bibliotecas de cada projeto e evita o clássico "funciona na minha máquina".

```
python -m venv .venv
.venv\Scripts\activate        # Windows
source .venv/bin/activate     # Linux e macOS
pip install requests
```

Use um editor que entenda Python, com realce de sintaxe, formatação automática e execução de testes. Mais importante que o editor é o hábito: um projeto, uma pasta, um ambiente virtual e um arquivo `requirements.txt` com as dependências.

2. O essencial da linguagem

Python foi desenhado para ser lido. A indentação faz parte da sintaxe, os nomes são em minúsculas com sublinhado e quase tudo é um objeto que você pode inspecionar no terminal interativo.

Tipos que você vai usar todo dia

- **Números**: `int` para inteiros e `float` para valores com casas decimais. Para dinheiro, prefira `Decimal`.
- **Textos**: `str`, com f-strings para montar mensagens: `f"Olá, {nome}"`.
- **Listas**: coleções ordenadas e mutáveis, como `[3, 1, 2]`.
- **Dicionários**: pares de chave e valor, como `{"cidade": "Pedra Alta", "habitantes": 4200}`.
- **Conjuntos**: elementos únicos, ótimos para remover duplicados.

Laços e decisões

O laço `for` percorre qualquer coleção. Combine-o com `enumerate` quando precisar do índice e com `zip` para andar em duas listas ao mesmo tempo.

```
vendas = {"jan": 120, "fev": 95, "mar": 143}
for mes, total in vendas.items():
    situacao = "acima" if total >= 100 else "abaixo"
    print(f"{mes}: {total} ({situacao} da meta)")
```

Funções pequenas e com nome bom

Uma função deve fazer uma coisa só e ter um nome que diga qual. Se você precisa de um comentário para explicar o que um bloco faz, provavelmente ele quer virar uma função.

```
def media(valores: list[float]) -> float:
    if not valores:
        raise ValueError("lista vazia")
    return sum(valores) / len(valores)
```

3. Trabalhando com arquivos

O módulo `pathlib` trata caminhos como objetos e funciona igual no Windows e no Linux. Use sempre o bloco `with`, que fecha o arquivo mesmo quando algo dá errado no meio do caminho.

```
from pathlib import Path
import csv

pasta = Path("relatorios")
with open(pasta / "vendas.csv", encoding="utf-8", newline="") as f:
    for linha in csv.DictReader(f, delimiter=";"):
        print(linha["produto"], linha["quantidade"])
```

Exercício

Escreva um script que leia todos os arquivos CSV de uma pasta, some a coluna de quantidade por produto e grave o resultado num novo arquivo `resumo.csv`, ordenado do produto mais vendido para o menos vendido.

4. Automatizando tarefas

Automatizar é transformar uma sequência de cliques em um comando. Comece pelas tarefas que você repete toda semana e que seguem sempre os mesmos passos: renomear arquivos, juntar planilhas, baixar relatórios, enviar o mesmo aviso para uma lista de pessoas.

```
from datetime import date
from pathlib import Path

for arquivo in Path("fotos").glob("*.jpg"):
    novo = arquivo.with_name(f"{date.today():%Y-%m-%d}_{arquivo.name}")
    arquivo.rename(novo)
```

Antes de rodar um script que altera arquivos, rode uma versão que só imprime o que faria. Esse "modo ensaio" evita surpresas e custa duas linhas de código.

Agendando a execução

No Windows, o Agendador de Tarefas executa scripts em horários fixos; no Linux e no macOS, o `cron` faz o mesmo papel. Registre a saída do script num arquivo de log para saber o que aconteceu enquanto você não estava olhando.

5. Boas práticas

Código bom é código que você entende daqui a seis meses. Algumas regras simples ajudam muito:

1. Siga o guia de estilo PEP 8 e deixe um formatador automático cuidar dos espaços.
2. Use anotações de tipo nas funções públicas; elas documentam e ajudam o editor a achar erros.
3. Escreva testes com `pytest` para as funções que fazem contas ou transformam dados.
4. Trate erros esperados com mensagens claras e deixe os inesperados aparecerem.
5. Guarde senhas e chaves em variáveis de ambiente, nunca no código.

```
# test_media.py
from calculos import media

def test_media_simples():
    assert media([2, 4, 6]) == 4
```

> *Automatizar não é sobre escrever menos código. É sobre errar menos e pensar mais no que importa.*

Próximos passos

Com o que está neste guia, você já consegue resolver problemas reais. Os próximos passos naturais são aprender a consultar APIs com a biblioteca `requests`, analisar dados com `pandas` e publicar pequenas aplicações web. Escolha um problema do seu trabalho, resolva com Python e só então procure a próxima ferramenta: a curiosidade guiada por um problema concreto ensina mais do que qualquer lista de cursos.